



udp UNIVERSIDAD
DIEGO PORTALES

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

COMUNICACIONES DIGITALES

Simulación de Canal AWGN y Cálculo de BER

Autores:

Joaquín Moya, Álvaro Valdebenito, José Avello

Profesor:

Marcos Fantoal

8 de junio de 2026

Índice

1. Introducción	2
2. Metodología	2
2.1. Actividades previas y bloques utilizados	2
2.2. Procedimiento experimental	3
2.3. Transmisor y modulación	3
2.4. Modelo del canal AWGN	4
2.5. Receptor y cálculo de la BER	4
3. Resultados y Análisis	5
3.1. Comportamiento de la constelación BPSK	5
3.2. Comparación entre BPSK, QPSK y 8-PSK	6
4. Conclusión	6
5. Referencias	7

1. Introducción

El presente informe detalla el diseño, implementación y análisis de un sistema de comunicaciones digitales en banda base utilizando GNU Radio. El objetivo principal de la experiencia es transmitir símbolos BPSK, QPSK y 8-PSK a través de un canal con Ruido Blanco Gaussiano Aditivo (AWGN) y evaluar el desempeño del sistema mediante la Tasa de Error de Bit (BER).

La BER permite comparar la cantidad de bits erróneos respecto del total de bits transmitidos. En este laboratorio se modifica la relación de energía por bit a densidad espectral de ruido (E_b/N_0) entre 1 y 11 dB, manteniendo constante la energía por bit y variando solamente la amplitud del ruido. De esta forma se observa cómo la separación entre símbolos y el orden de modulación influyen directamente en la probabilidad de error.

2. Metodología

El sistema fue diseñado mediante un diagrama de bloques en GNU Radio, dividiéndose en tres etapas: transmisor (TX), canal AWGN y receptor (RX). La Fig. 1 muestra la implementación general utilizada para realizar las mediciones.

2.1. Actividades previas y bloques utilizados

Un **byte pack** agrupa varios bits individuales dentro de una palabra de tipo byte. En cambio, un **byte unpack** realiza la operación inversa: toma un byte y separa sus bits para poder analizarlos o compararlos bit a bit. Esta operación es necesaria para calcular correctamente la BER, ya que el bloque de comparación debe recibir secuencias binarias equivalentes.

Los bloques principales utilizados fueron los siguientes:

- **Random Source:** genera la secuencia aleatoria de datos. Se configuró para producir enteros entre 0 y $M - 1$, donde M corresponde al orden de la modulación.
- **Chunks to Symbols:** convierte cada entero de entrada en un símbolo complejo de la constelación seleccionada.
- **Noise Source:** genera muestras de ruido gaussiano complejo. Su amplitud fue controlada por la variable E_b/N_0 .
- **Constellation Decoder:** decide cuál símbolo ideal fue recibido, escogiendo el punto de la constelación más cercano a la muestra ruidosa.
- **BER:** compara la secuencia transmitida con la recibida y entrega el valor de error en escala logarítmica, es decir, $\log_{10}(BER)$.

- **QT GUI Number Sink:** permite visualizar numéricamente la BER medida durante la simulación.

Para el objeto **gr::digital::constellation**, los métodos solicitados cumplen las siguientes funciones: **arity()** entrega el número de símbolos de la constelación; **points()** retorna las coordenadas complejas de dichos símbolos; **base()** indica la base usada para representar los símbolos; y **bits_per_symbol()** entrega la cantidad de bits asociados a cada símbolo, equivalente a $\log_2(M)$.

2.2. Procedimiento experimental

El procedimiento seguido para reproducir la simulación fue:

1. Definir la variable M con los valores 2, 4 y 8 para simular BPSK, QPSK y 8-PSK, respectivamente.
2. Configurar el bloque **Random Source** para generar símbolos enteros en el rango $[0, M - 1]$.
3. Crear el objeto de constelación correspondiente y conectarlo al bloque **Chunks to Symbols** para realizar el mapeo I/Q.
4. Sumar a la señal modulada una fuente de ruido gaussiano mediante el bloque **Add**.
5. Controlar la amplitud del ruido con un *QT GUI Range*, realizando un barrido de E_b/N_0 desde 1 hasta 11 dB.
6. Decodificar los símbolos recibidos con **Constellation Decoder**.
7. Desempaquetar los símbolos transmitidos y recibidos con **Unpack K Bits**, usando $K = \log_2(M)$.
8. Comparar ambas secuencias en el bloque **BER** y registrar el valor obtenido para cada E_b/N_0 .

El fragmento lógico equivalente usado para controlar la amplitud del ruido fue:

```
k = log2(M)
noise_voltage = sqrt(1/(2*k*10**((EbN0_dB/10))))
```

2.3. Transmisor y modulación

La generación de datos se realizó mediante el bloque **Random Source**, configurado para generar números enteros entre 0 y $M - 1$. Estos datos son entregados al bloque **Chunks to Symbols**, el cual mapea cada símbolo a un punto específico del plano complejo I/Q.

Para asegurar una sincronización perfecta entre el transmisor y el receptor, se definió un **Constellation Object** con las posiciones exactas de los símbolos. En BPSK se

utilizaron los puntos -1 y 1 ; en QPSK cuatro puntos equiespaciados; y en 8-PSK ocho puntos distribuidos sobre una circunferencia unitaria.

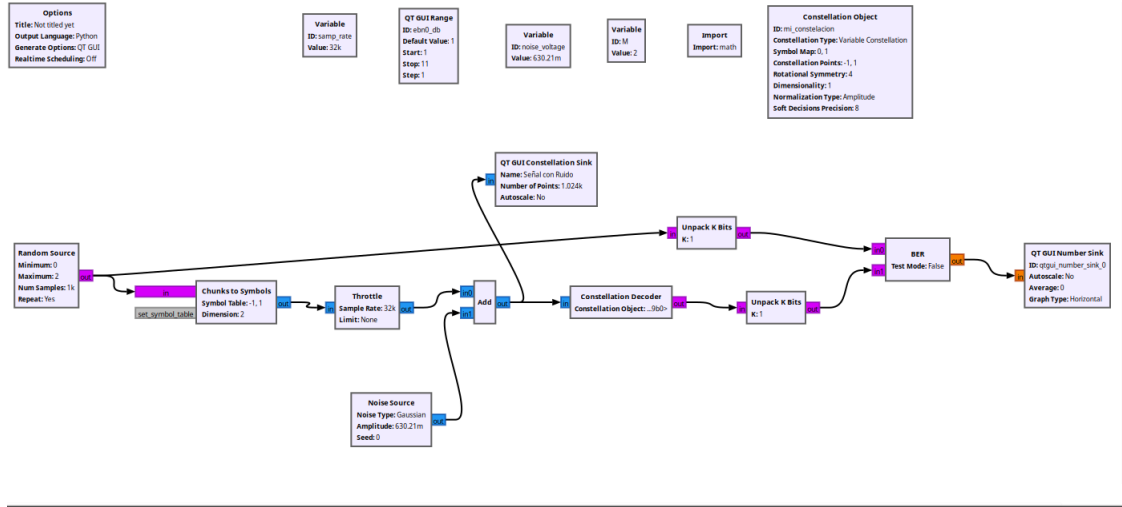


Figura 1: Diagrama de bloques del sistema implementado en GNU Radio para transmitir, añadir AWGN, decodificar y calcular BER.

2.4. Modelo del canal AWGN

El canal de transmisión introduce distorsión a la señal modulada simulando las condiciones del espacio libre o cables. Esto se logró sumando la señal original con una fuente de ruido gaussiano mediante el bloque **Add**.

La amplitud del ruido no es un valor estático, sino que depende de la relación E_b/N_0 en decibelios. Para lograr una actualización en tiempo real mediante un control deslizante (*QT GUI Range*), se inyectó la siguiente fórmula matemática en la amplitud del bloque **Noise Source**:

$$V_{ruido} = \sqrt{\frac{1}{2 \cdot \log_2(M) \cdot 10^{\frac{E_b/N_0}{10}}}} \quad (1)$$

Esta ecuación asegura que, al variar la barra de E_b/N_0 , el voltaje de ruido se ajuste instantáneamente, permitiendo observar la dispersión de los puntos en el diagrama de constelación de forma interactiva.

2.5. Receptor y cálculo de la BER

Una vez que la señal ruidosa pasa por el canal, ingresa a la etapa de recepción. El bloque **Constellation Decoder** toma las muestras complejas recibidas y evalúa a

qué símbolo ideal de la constelación original se parecen más, tomando una decisión basada en la distancia euclidiana.

Para calcular los errores introducidos por el ruido, los símbolos recuperados se deben comparar bit a bit con los transmitidos. Para ello, se utilizan bloques **Unpack K Bits**, configurados matemáticamente para desempaquetar $\log_2(M)$ bits por símbolo.

Finalmente, el bloque **BER** compara la secuencia original de la fuente con la secuencia demodulada. El resultado entregado por este bloque es logarítmico ($\log_{10}(\text{BER})$), por lo que un valor de $-1,2$ equivale a una tasa de error aproximada de $10^{-1,2} = 0,063$, es decir, 6,3 %.

3. Resultados y Análisis

Se realizó un barrido del parámetro E_b/N_0 desde 1,0 dB hasta 11,0 dB utilizando un sumidero de constelación (*QT GUI Constellation Sink*) para observar el impacto físico del ruido sobre los símbolos.

3.1. Comportamiento de la constelación BPSK

Con un nivel bajo de E_b/N_0 (por ejemplo, cercano a 1 dB), la potencia del ruido es comparable con la potencia de la señal. En la Fig. 2, esto se visualiza como dos nubes de puntos altamente dispersas en torno a -1 y 1 . Cuando la dispersión hace que puntos asociados al símbolo 1 crucen la frontera de decisión hacia el lado negativo, o viceversa, el decodificador comete un error y la BER aumenta.

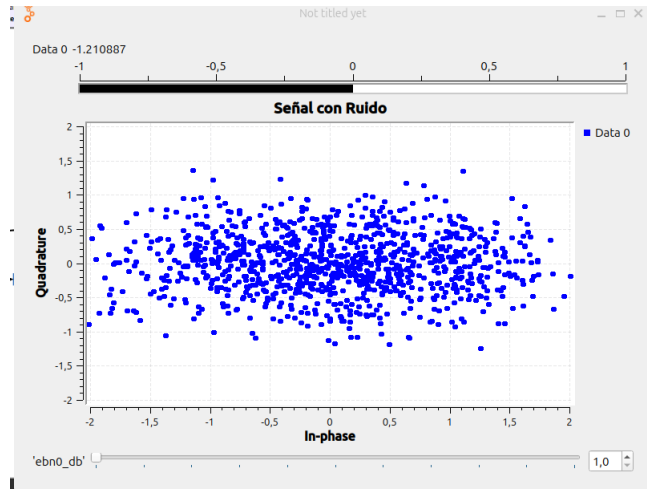


Figura 2: Constelación BPSK con bajo E_b/N_0 : las muestras se dispersan y aumenta la probabilidad de cruzar la frontera de decisión.

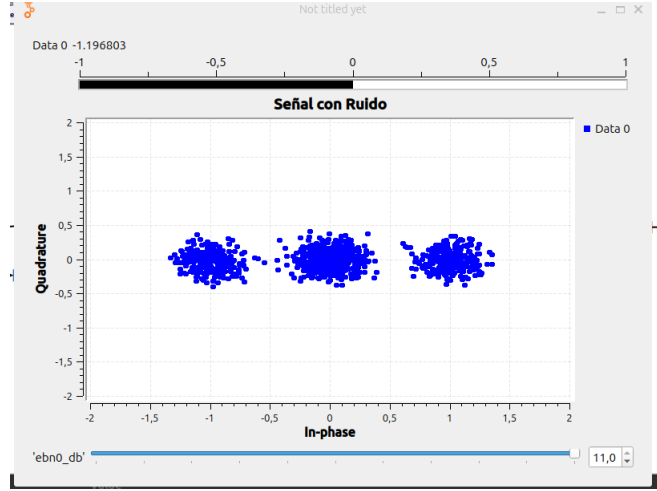


Figura 3: Constelación BPSK con mayor E_b/N_0 : las muestras se concentran cerca de los puntos ideales y disminuye la BER.

A medida que el E_b/N_0 se acerca a 11,0 dB, la potencia de la señal domina sobre el ruido. Las nubes se contraen hasta ubicarse cerca de los símbolos ideales, lo que reduce la probabilidad de cruzar la frontera de decisión y disminuye la tasa de error a valores muy pequeños.

3.2. Comparación entre BPSK, QPSK y 8-PSK

Gracias a la parametrización del sistema con la variable M , el cambio hacia modulaciones de orden superior se realiza de manera transparente. Al pasar a $M = 4$ (QPSK) y $M = 8$ (8-PSK), se aumenta la tasa de transferencia de datos, enviando 2 y 3 bits por símbolo respectivamente.

Sin embargo, al mantener la misma potencia de señal y añadir más símbolos al mismo plano complejo, la distancia entre ellos disminuye. Esto hace que modulaciones como 8-PSK sean significativamente más susceptibles al ruido AWGN que BPSK, requiriendo un E_b/N_0 mucho mayor para alcanzar la misma tasa de error.

4. Conclusión

El desarrollo de este laboratorio permitió comprobar la relación inversa entre E_b/N_0 y BER en un sistema digital de banda base. Al aumentar E_b/N_0 desde 1 hasta 11 dB, la amplitud del ruido disminuyó y los puntos de la constelación se concentraron alrededor de sus posiciones ideales, reduciendo la cantidad de decisiones incorrectas en el receptor.

La configuración más robusta fue BPSK, porque sus dos símbolos poseen la mayor separación posible en la constelación. QPSK permite transmitir 2 bits por símbolo

y mantiene una BER teórica similar a BPSK bajo las condiciones ideales consideradas, por lo que resulta conveniente cuando se busca mayor eficiencia espectral sin aumentar demasiado el error. En cambio, 8-PSK transmite 3 bits por símbolo, pero fue la alternativa más sensible al AWGN: para un mismo E_b/N_0 presentó una BER mayor debido a la menor separación entre símbolos.

En consecuencia, la elección de modulación depende del compromiso entre velocidad y confiabilidad. Si el canal es ruidoso o se requiere minimizar errores, BPSK es la opción más segura. Si el canal posee una relación E_b/N_0 suficientemente alta, QPSK u 8-PSK pueden aprovechar mejor el ancho de banda, aunque 8-PSK exige un canal más limpio para evitar solapamiento entre las nubes de ruido.

5. Referencias

- W. Tomasi, *Sistemas de comunicaciones electrónicas*. México: Pearson Educación, 2003.
- L. Couch, *Sistemas de comunicaciones digitales y analógicos*. México: Pearson/Prentice Hall, 2008.
- Documentación de GNU Radio para los bloques *Random Source*, *Chunks to Symbols*, *Noise Source*, *Constellation Decoder* y *BER*.
- J. Avello, “Comunicaciones digitales – LAB_3,” GitHub repository. Disponible en: https://github.com/JoseAvello/comunicaciones-digitales/tree/main/LABORATORIO_4